Predicting machine failure from process parameters

A plain-language guide to the whole project: the data, every decision, the code, the results and the dashboard

The project in one paragraph

A milling machine runs 10,000 times. For each run we know its settings (speed, torque, temperatures, how worn the tool is) and whether it broke down. Only 339 runs (3.39%) failed. We built a model that reads the settings and estimates the **probability of failure**, explained **which settings cause failure**, and turned the model into **recommended safe operating ranges** for each product type. Everything is checked with honest, repeatable cross-validation, and a dashboard lets anyone explore it.

What	Result	In plain words
Main score (PR-AUC)	0.915 ± 0.006	Excellent at ranking risky runs above safe ones (a random guess scores 0.034)
Failures caught	89.4%	At the chosen alarm setting, about 9 in 10 failures are flagged in advance
Alarms that are real	52.2%	About 1 in 2 alarms is a real failure; the other is a cheap extra inspection
Best model	Random Forest	Beat XGBoost on all 5 test folds, and beat Logistic Regression by a wide margin
Safe operating range	power 4.75–8 kW, tool change before ~200 min	Matches the machine's real physics, which the model was never told

How to use this guide

Sections 1–3 explain the problem and the data. Section 4 explains every decision and *why* it was made, which is the part you will be graded on. Sections 5–7 cover the results, explanations and recommendations. Section 8 walks through the code file by file, in the order you should review it. Section 9 shows how to read the dashboard. Section 11 is a cheat-sheet of likely questions with short answers. A glossary is at the end.

1. The problem

Factories lose money when machines break unexpectedly: production stops, parts are scrapped, repairs are urgent. **Predictive maintenance** tries to see a failure coming from the machine's own readings, so that someone can intervene first (slow down, change the tool, inspect).

The project has three goals, which build on each other:

1. **Predict**: given a run's settings, how likely is a failure? (a *classification* problem: failure yes/no)
2. **Explain**: which settings drive that risk, and how?
3. **Recommend**: which settings keep the risk low? (an *optimisation* layer on top of the model)

> **Why this is harder than it sounds**
> Failures are **rare**: only 3.39% of runs. A lazy model that always says "no failure" is right 96.6% of the time and completely useless. Most of the decisions in this project are about not being fooled by this.

2. The dataset and its three traps

AI4I 2020 Predictive Maintenance Dataset (UCI repository, id 601). It is *synthetic*: generated to mimic a real milling machine, with failures produced by known physical rules. That is why we can later check our model against those rules (Section 7).

Column	Meaning	Used as
UID, Product ID	Row number and serial number	Dropped (just identifiers)
Type	Product quality: L (low, 60%), M (medium, 30%), H (high, 10%)	Input
Air temperature [K]	Room temperature, around 300 K (27 °C)	Input
Process temperature [K]	Temperature at the machining process, about 10 K above air	Input
Rotational speed [rpm]	How fast the spindle turns	Input
Torque [Nm]	Twisting force on the tool	Input
Tool wear [min]	Minutes the current tool has been used	Input
Machine failure	1 if the run failed, else 0	**Target** (what we predict)
TWF, HDF, PWF, OSF, RNF	Which kind of failure happened (5 yes/no flags)	**Dropped: leakage**

The basic checks (notebook 01): 10,000 rows, 14 columns, **no missing values, no duplicate rows**. One difference from the brief: the downloaded copy names the columns without units ("Air temperature" instead of "Air temperature [K]"). The loader handles both versions.

Trap 1: severe class imbalance

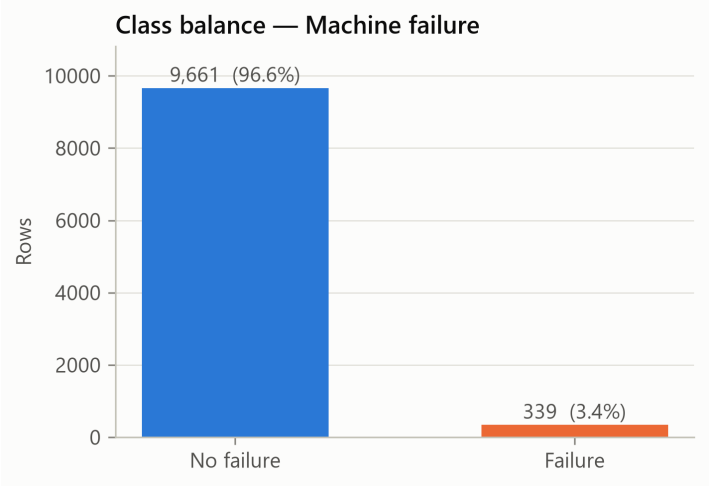


Figure 1. 9,661 healthy runs vs 339 failures, a ratio of 28.5 to 1.

Consequence: **accuracy is meaningless here**, and the model must be pushed to care about the rare class. This drives the choice of metric (4.2) and the imbalance strategies we tested (4.4).

Trap 2: leakage (the answer hidden in the inputs)

The five flags TWF, HDF, PWF, OSF and RNF say *which type* of failure happened. They are only known **after** the failure, and "Machine failure" is basically "any of these flags is 1". If we fed them to the model, it would score about 100% while learning nothing about the process.

> **Proof that it is leakage**
> Without any model at all, the rule "predict failure if any flag is 1" already catches **97.3%** of failures at **94.8%** precision. A real model never sees these flags when it is used, so they must be removed.

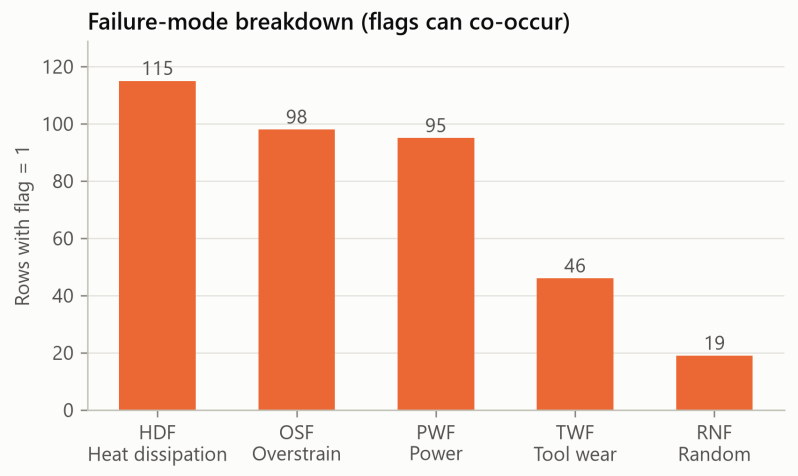


Figure 2. How often each failure type occurs. Some runs have two at once.

Trap 3: small quirks in the labels

- 18 runs have the "random failure" flag set but are **not** labelled as failures, and 9 failures have **no** flag at all. We kept the target exactly as given (we do not "fix" labels), and noted it.
- Product ID's first letter is the same as Type, so it adds nothing. It was dropped with the IDs.

3. What the data showed (exploratory analysis)

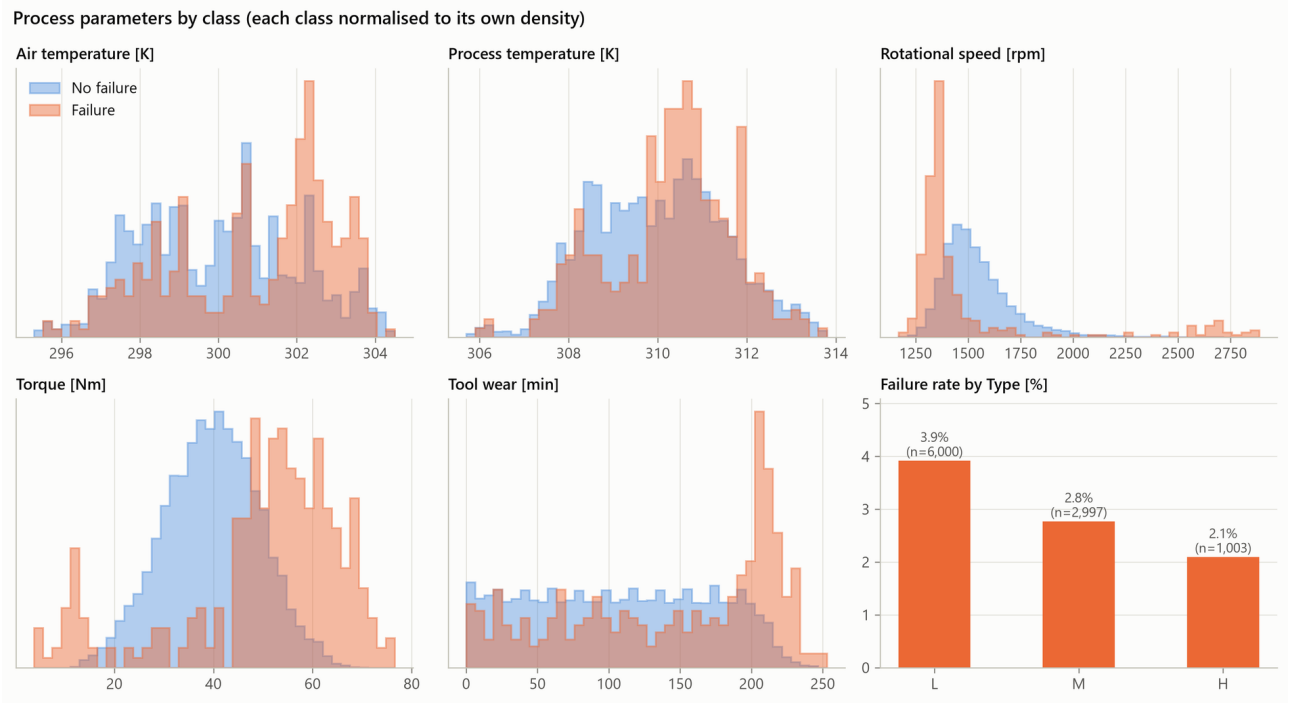


Figure 3. Each parameter, healthy (blue) vs failed (orange). Each class is scaled to its own size so the small failure group is visible. Last panel: failure rate by product Type.

- Failures happen at **higher torque** (median 53.7 vs 39.9 Nm), **lower speed** (1,365 vs 1,507 rpm), **more tool wear** (165 vs 107 min, with a spike around 200–240 min) and slightly **higher air temperature**.
- Failure rate falls from L (3.9%) to M (2.8%) to H (2.1%). That ordering is why Type is encoded as L=0 < M=1 < H=2.
- Speed and torque are **tightly linked** (correlation −0.92): fast runs have low torque and vice versa. Air and process temperature are also linked (0.86).
- Simple correlations with failure are weak (at most 0.17) even though the groups clearly differ. So the relationships are **non-linear** (thresholds and combinations), which favours tree models over a straight-line model.

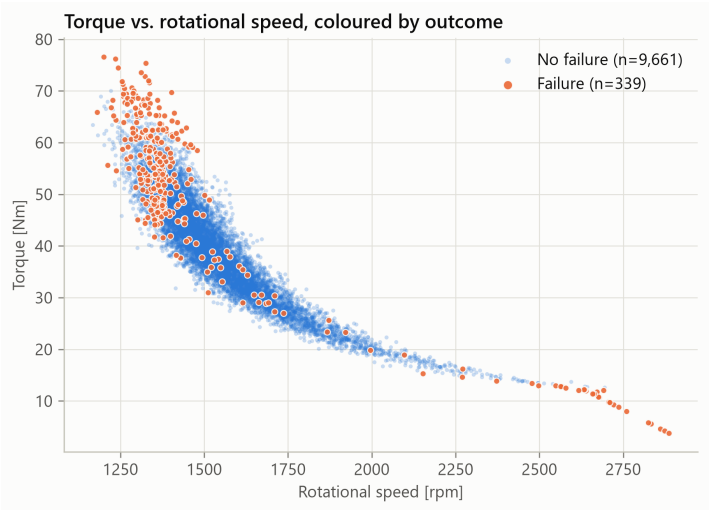


Figure 4. Failures (orange) cluster at **both** *ends of the operating band: high torque at low speed, and low torque at high speed. Both ends are power problems (power = torque × speed).*

4. Every key decision, and why

Each decision below says **what** we did, **why**, and **where** in the code it lives. The three graded decisions are marked **[GRADED]**, and each has a comment in the code at the exact spot.

4.1 Remove the leakage columns [GRADED]

What: the five failure-type flags and the two ID columns are removed from the inputs. The model sees exactly six inputs: Type plus the five process parameters.

Why: the flags are outcomes, not inputs (Section 2). The IDs are row numbers with no physical meaning.

Where: `src/data.py`, function `get_X_y()`. This is the *only* place the input table is built, and a safety check (`assert`) stops the program if a flag ever sneaks back in.

4.2 Choose the right score: PR-AUC, not accuracy [GRADED]

First, the two ideas every score is built from. Imagine the model raises an "alarm" for runs it thinks will fail:

Term	Question it answers	Example
Recall	Of all real failures, how many did we catch?	Catch 303 of 339 failures, so recall = 89%
Precision	Of all alarms raised, how many were real failures?	583 alarms, 303 real, so precision = 52%
F1	One number balancing the two	Their harmonic mean
Accuracy	Of all runs, how many did we label correctly?	**Misleading**: 96.6% for a model that never raises an alarm

A model doesn't say yes/no directly. It outputs a **probability**, and we choose a **threshold** above which we raise the alarm. Lower the threshold and you catch more failures (recall up) but raise more false alarms (precision down).

> **PR-AUC (average precision): our main score**
> Draw precision against recall for **every possible threshold**. That is the PR curve. PR-AUC is the area under it: 1.0 is perfect, and **a random guess scores only the failure rate, 0.034**. It looks only at how well the *failure class* is handled, and it doesn't depend on picking a threshold first.

Why not ROC-AUC? ROC-AUC measures something similar but divides false alarms by the 9,661 healthy runs, so hundreds of false alarms barely dent it. Our logistic regression scores a flattering 0.92 ROC-AUC but only 0.50 PR-AUC. We report ROC-AUC, but never choose by it.

Where: `src/cv.py` (comment "GRADED DECISION #3"). Accuracy is shown in every table, labelled "misleading", and never used to pick anything.

4.3 Test honestly: stratified 5-fold cross-validation

A model must be judged on data it has **never seen**. Instead of one train/test split (which could be lucky or unlucky), the data is cut into 5 equal parts (**folds**). We train on 4, test on the 5th, and repeat 5 times so every run is tested exactly once.

- **Stratified**: every fold keeps the same 3.39% failure rate, so no fold is accidentally failure-free.
- Every score is reported as **mean ± standard deviation** over the 5 folds, never a single number. The ± shows how stable it is.
- Every model uses **the same 5 folds** (fixed random seed 42). Comparisons are therefore fair ("paired"), and the results reproduce exactly. We re-ran everything and got identical numbers.
- **Out-of-fold (OOF) predictions**: gluing together each fold's test predictions gives one honest prediction per run, from a model that never trained on it. The curves and confusion matrices use these.

4.4 Handle the imbalance: three strategies compared, SMOTE inside the fold [GRADED]

With 28 healthy runs per failure, a model can get lazy and under-predict failure. We compared three remedies in the same test setup:

Strategy	Idea in one line
(a) None	Train on the data as it is (the baseline).
(b) Class weighting	Tell the model a mistake on a failure costs ~28× more than a mistake on a healthy run (`class_weight='balanced'`; XGBoost: `scale_pos_weight`).
(c) SMOTE	Create extra, synthetic failure examples so both classes are equally common during training.

How SMOTE works, simply

Pick a real failure, find its nearest failure neighbours, and invent a new point somewhere on the line between them. Repeat until there are as many failures as healthy runs. We use **SMOTENC**, a variant that handles the categorical Type column properly (plain SMOTE would invent nonsense like "Type 0.37").

The rule that matters: resample ONLY inside each training fold

SMOTE must run **after** the split and **only on the 4 training folds**. The test fold stays 100% real. If you apply SMOTE to the whole dataset first, near-copies of test failures leak into training, and the test fold is half fake. We demonstrated this on purpose: doing it wrong gives a fake **PR-AUC of 0.999**, while doing it right gives **0.872**.

How it is guaranteed: SMOTE is a step inside an `imblearn` Pipeline. That kind of pipeline only resamples when the model is being *trained*, and skips resampling when it *predicts*. The cross-validation code only ever trains on the training folds. **Where:** `src/pipeline.py` (comment "GRADED DECISION #2").

Result (honest): neither weighting nor SMOTE improved PR-AUC for any model. They push predicted probabilities upward, which trades precision for recall at the default threshold. Moving the threshold does exactly that trade, more transparently (4.8). The best model uses no resampling. This is a legitimate, defensible finding, not a failure.

4.5 Add physics-based features

Each failure type follows a physical mechanism, so we computed the quantities those mechanisms depend on and handed them to the model directly:

New feature	Formula	Linked to
Power [W]	torque × rotational speed × 2π/60	Power failure: too little or too much power
Temperature difference [K]	process temperature − air temperature	Heat-dissipation failure: heat can't escape
Overstrain [min·Nm]	tool wear × torque	Overstrain failure: a worn tool under heavy load

Ablation (with vs without these features, same folds): PR-AUC rose by **+0.08 to +0.14 for every tree model**, better on 5 of 5 folds each time. Trees split on one variable at a time, so a boundary like "power (torque × speed) above 9 kW" is a curve they can only approximate with many small steps; given the product directly, they need just one. **Where:** `src/features.py`.

4.6 Preprocessing

- **Type** → ordinal numbers L=0, M=1, H=2 (justified by the falling failure rate, Section 3).
- **Numbers** → standardised (rescaled to mean 0, spread 1). Logistic regression needs this. Trees don't, but SMOTE does: it measures distances, and unscaled speed (spread ~180) would swamp temperature (spread ~2). Rescaling never changes a tree's decisions, so we scale for every model and keep one simple pipeline.
- Both are done by a `ColumnTransformer` inside the pipeline, so they are learned from training folds only.

4.7 Three models, from simple to powerful

Model	How it thinks, simply	PR-AUC (best strategy)
Logistic Regression	Draws one straight boundary through the data. The simple baseline.	0.495 ± 0.048
Random Forest	400 decision trees, each trained on random samples; they vote.	0.905 ± 0.008
XGBoost	Trees built one after another, each fixing the previous ones' mistakes.	0.875 ± 0.017

Grid tested: 3 models × 3 imbalance strategies × with/without physics features = **18 configurations**, all on the same folds.

4.8 Tuning without fooling ourselves: nested cross-validation

Models have settings ("hyper-parameters", e.g. how deep each tree may grow). If you try many settings and report the best score on the same folds, the score is optimistic, because you picked the lucky one. **Nested CV** fixes this: inside each training fold, a *second* cross-validation picks the settings, and the untouched test fold then scores the result.

- Random Forest and XGBoost each got the **same tuning budget** (20 random candidates).
- Tuned Random Forest: **0.915 ± 0.006** (+0.010). The inner search chose the **same settings in all 5 folds** (max depth 8, 80% of features per split, at least 2 samples per leaf), which means the choice is stable.
- Tuned XGBoost: no improvement (0.872). Its inner scores looked better than its test scores, which is exactly the optimism nested CV exists to expose.

4.9 Choosing the alarm threshold

The default threshold of 0.5 catches 82% of failures with almost no false alarms (precision 98%). In maintenance, **a missed failure costs far more than an extra inspection**, so we decided *before* looking at results to catch at least 90% of failures, using the highest threshold that still achieves that (the best precision at that recall).

- Chosen threshold: **0.056**, stable across folds (0.052–0.064).
- It was tuned with nested CV too: chosen inside the training folds, then scored on untouched test folds.
- Trade-off: **26 more failures caught** (277 → 303 of 339) for **275 more false alarms** (5 → 280, i.e. 2.9% of healthy runs). Precision drops from 98% to 52%.
- Accuracy goes *down* (99.3% → 96.8%) while the model gets better at its real job, one more reason accuracy is ignored.

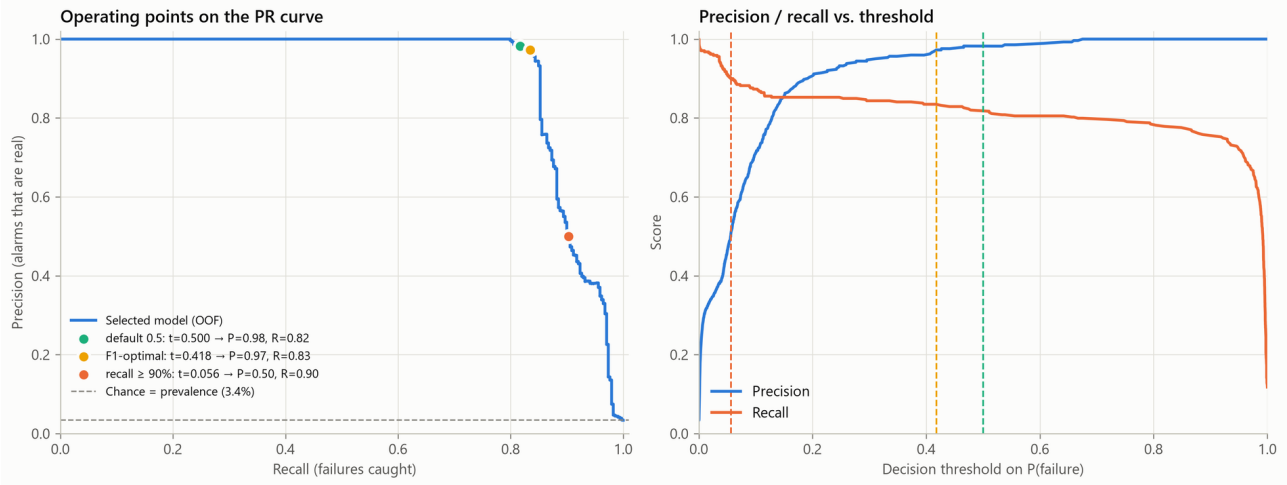


Figure 5. Left: the PR curve with three possible thresholds marked. Right: how precision (blue) and recall (orange) move as the threshold changes. Past about 85% recall, every extra failure caught costs many false alarms.

5. Results

PR-AUC (mean ± std, 5 folds)	No resampling	Class weighting	SMOTE (in-fold)
Logistic Regression	0.495 ± 0.048	0.452 ± 0.050	0.431 ± 0.060
Random Forest	**0.905 ± 0.008**	0.892 ± 0.014	0.878 ± 0.009
XGBoost	0.872 ± 0.019	0.875 ± 0.017	0.872 ± 0.013
Random Forest, tuned (nested CV)	**0.915 ± 0.006**		

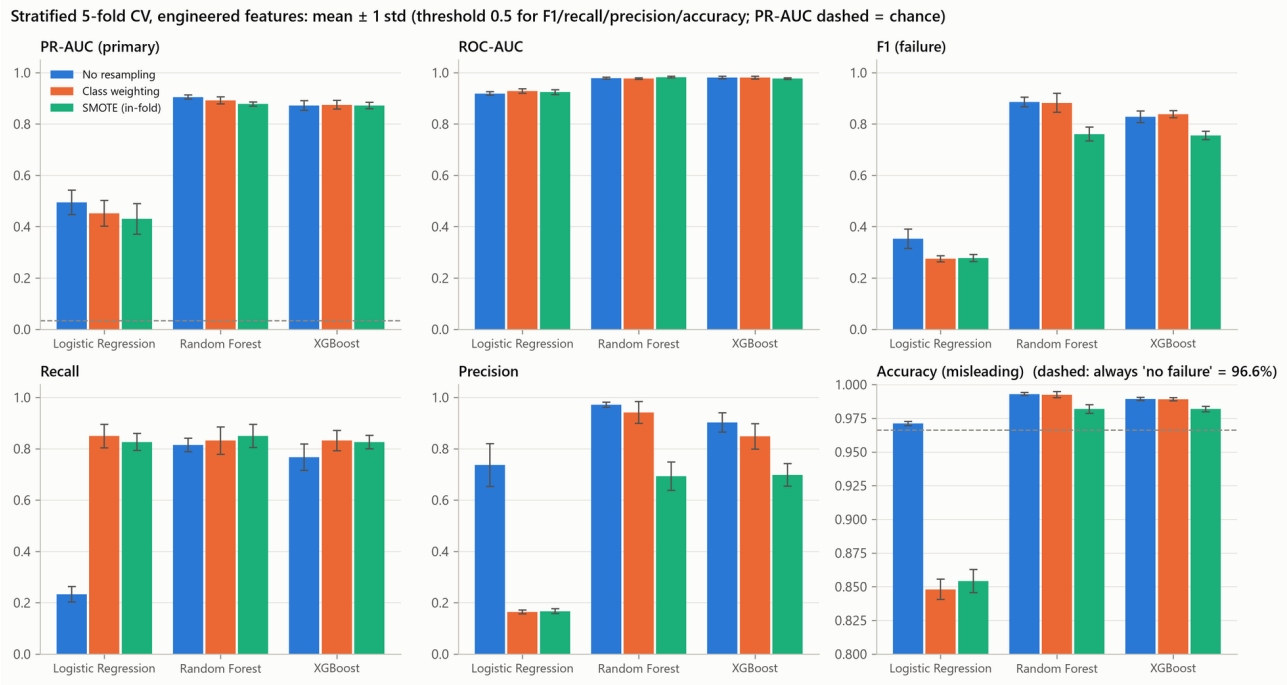


Figure 6. All six scores for all 9 model/strategy pairs (with physics features). Bars = mean, whiskers = ± 1 std. Note the accuracy panel: every model sits near the 96.6% "always say no" line.

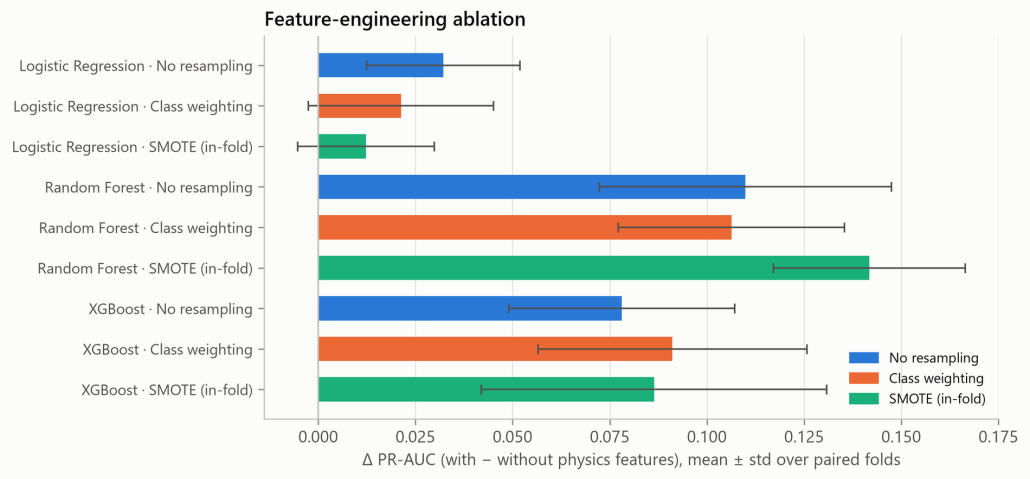


Figure 7. Gain from the physics features, per model and strategy, measured on paired folds.

Selected model at…	Recall	Precision	F1	Missed	False alarms
threshold 0.5	0.817 ± 0.026	0.982 ± 0.017	0.892	62	5
threshold 0.056 (chosen)	**0.894 ± 0.016**	**0.522 ± 0.036**	0.658	36	280

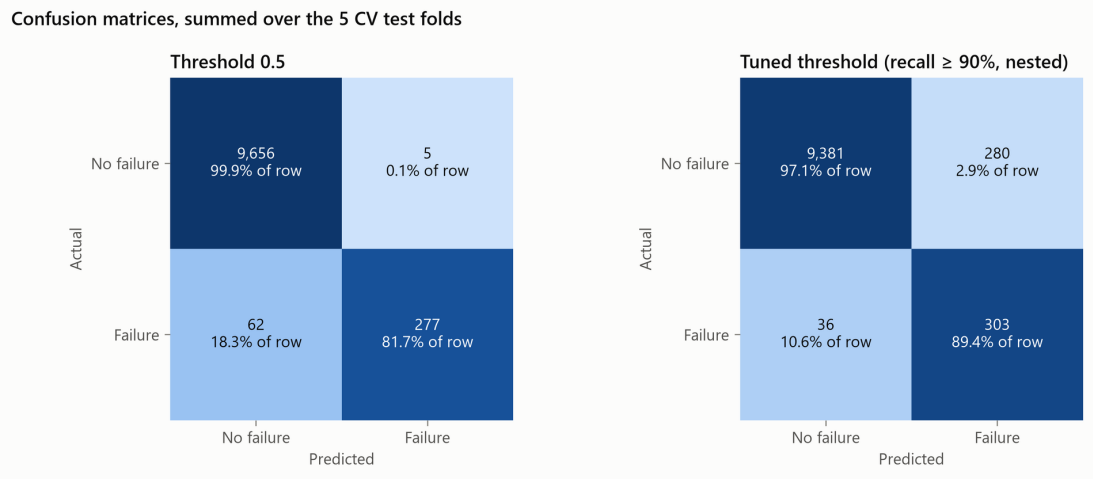


Figure 8. Confusion matrices summed over the 5 test folds. Rows = what really happened, columns = what the model said.

6. Explaining the model

6.1 Feature importance

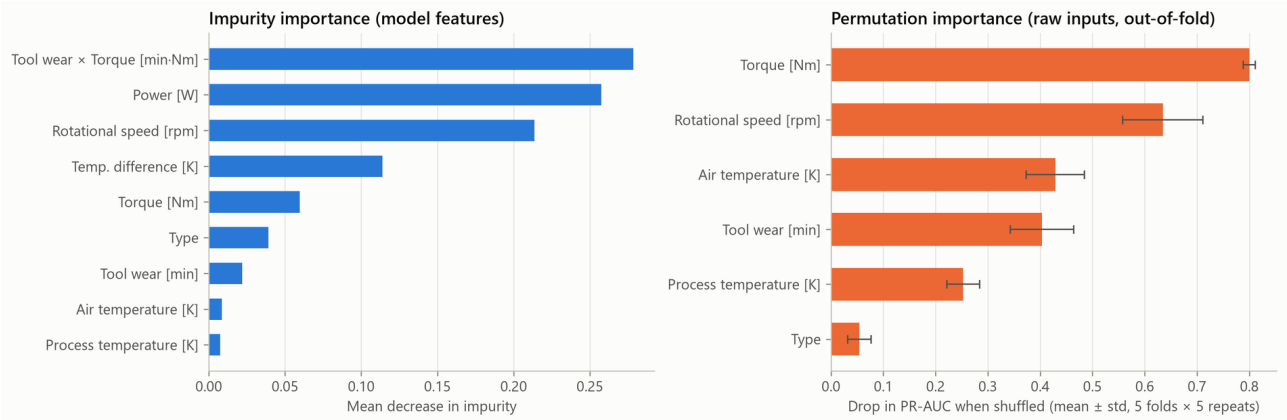


Figure 9. Left: how much each feature is used inside the forest. Right: how much the score drops when one raw input is randomly shuffled (measured on test folds). A bigger drop means more important.

Torque and speed matter most as raw inputs, mainly through the power and overstrain features built from them. Air temperature ranks high because it drives the temperature difference.

6.2 SHAP: why the model said what it said

> **SHAP in plain words**
> Start from the average failure risk (3.4%). For one run, SHAP splits the gap between that average and the run's prediction among the inputs: "tool wear × torque added +89 points, temperature difference took away 1 point…". The pieces always add up exactly to the prediction (we check this in code). Here the units are probability.

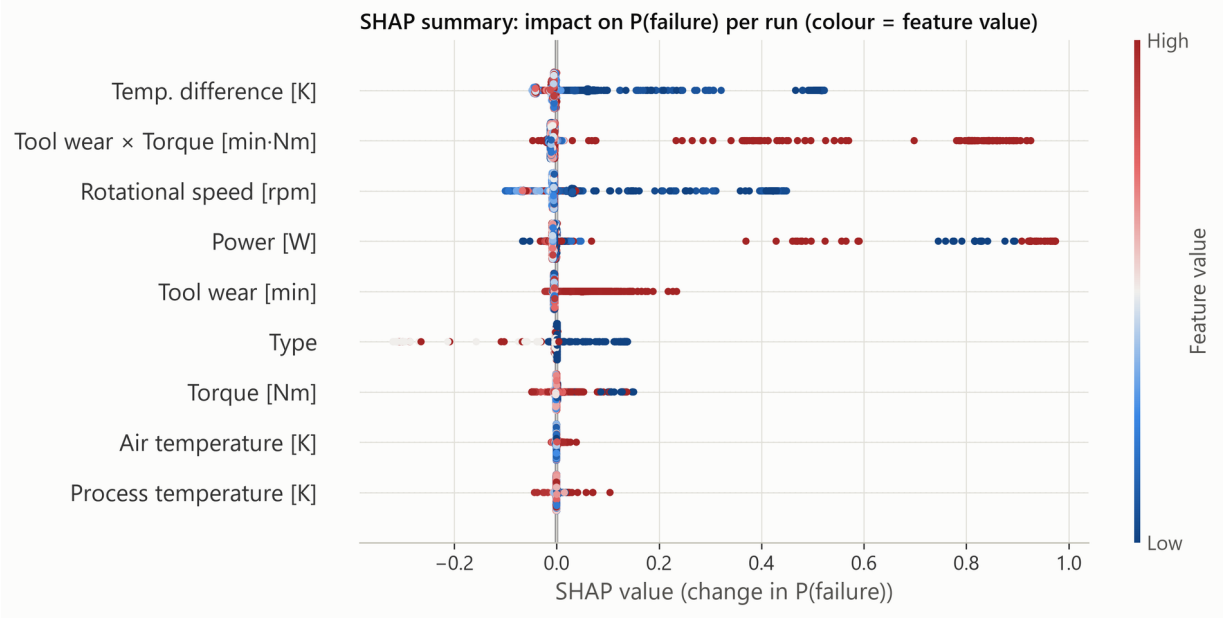


Figure 10. SHAP summary for all 10,000 runs. Each dot is a run; colour = the feature's value (red high, blue low); position = how much it pushed the risk up (right) or down (left).

- **Low temperature difference** (blue) pushes risk up: the heat-dissipation mechanism.
- **High overstrain** (red) pushes risk up strongly: the overstrain mechanism.
- **Power is two-sided**: both low and high power push risk up. A straight-line model can't express this, which explains why logistic regression lagged.
- **Tool wear** on its own has a moderate effect, only beyond about 200 min.

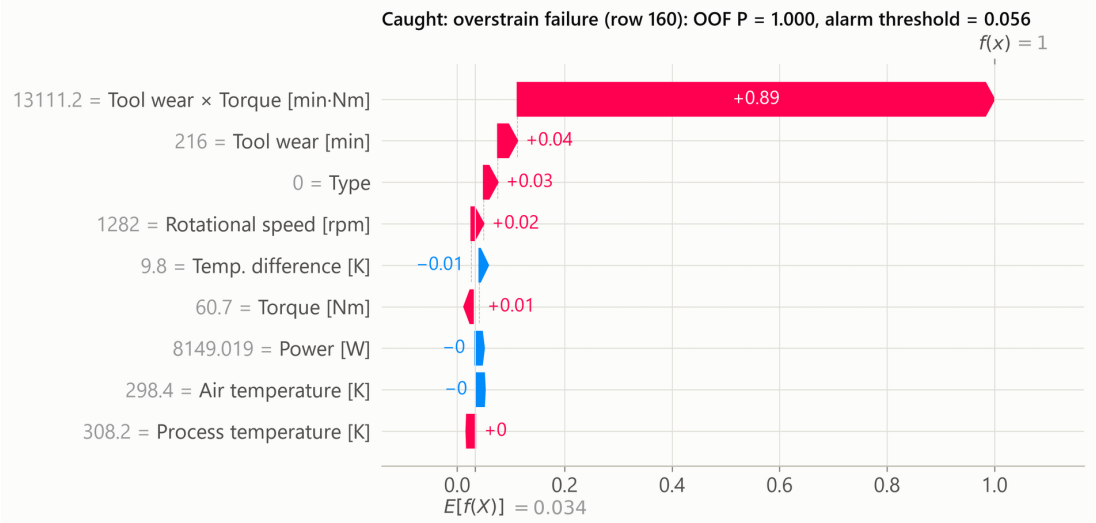


Figure 11. One caught overstrain failure: the wear × torque value (13,111, above the 11,000 limit for Type L) explains almost all of the prediction.

The notebook also shows a **missed** failure. It is one of the 9 failures with no failure flag, and its settings are unremarkable. Models that never saw it give it a 0.000 probability. No model built on these inputs could catch it, which is why recall can't reach 100%.

6.3 Partial dependence

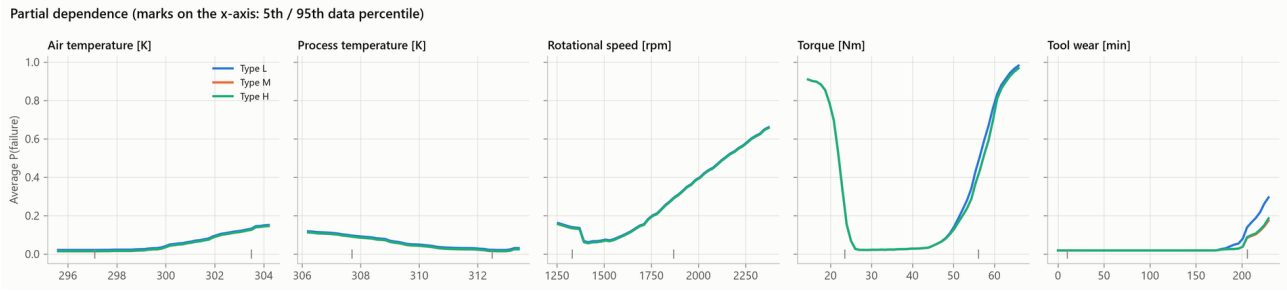


Figure 12. Average predicted risk as one parameter is varied for every run. Torque shows the two-sided power band; tool wear climbs after ~200 min.

Caution: the rising speed curve is partly an artefact. Setting every run to 2,300 rpm while keeping its own torque creates impossible combinations (speed and torque are linked). That is why the recommendations use the 2-D, data-supported analysis instead.

7. From model to recommendations

Goal: find settings that keep the failure risk low. The controllable settings are **speed and torque** (the operating point) and **tool wear** (set by when you change the tool). We predicted the risk on a fine grid of 141 × 151 speed/torque combinations, for every product Type and several tool-wear levels. In 2-D an exhaustive grid is exact and easy to check, so no fancy optimiser (such as Optuna) was needed.

Safeguard 1: only recommend what the data has seen

Speed and torque are linked, so about 90% of the speed × torque rectangle never occurs in the data (only 2,210 of 21,291 grid cells have real runs nearby). There, a tree model just guesses from its nearest edge. We only colour and recommend **data-supported** settings.

Safeguard 2: a risk budget, not the alarm threshold

The alarm threshold (0.056) answers "should this particular run be flagged?". A design range answers a different question: "which settings keep risk near normal?". Random tool failures make every run above ~200 min of wear about 5% risky, just below 0.056. Using the alarm threshold as a design limit would therefore approve 230 min of tool wear. So a setting counts as **safe if its risk is ≤ 1%** (typical healthy settings sit near 0.1%). Budgets between 0.5% and 2% give the same answers, so the choice is not fragile.

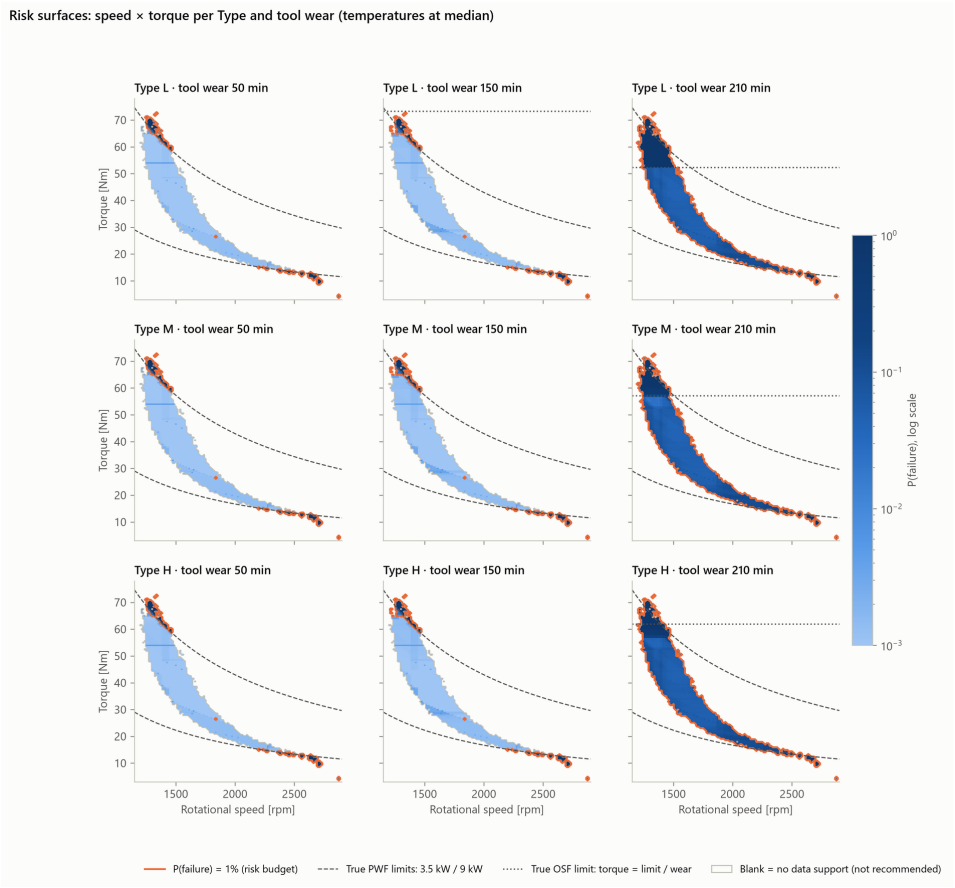


Figure 13. Predicted risk over speed × torque, for each Type (rows) at three tool-wear levels (columns). Only data-supported settings are coloured; the orange line marks 1% risk; dashed lines are the true power limits.

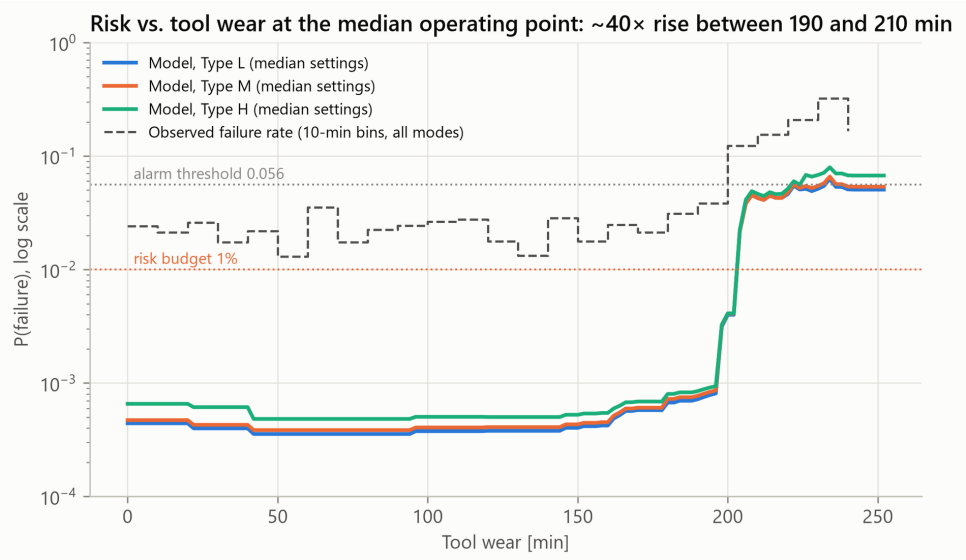


Figure 14. Risk vs tool wear at a typical setting. It rises about 40× between 190 and 210 min. Hence "change the tool before ~200 min".

Recommended operating ranges (normal temperatures; safe = risk ≤ 1% at every tool wear up to the limit):

Type	Change tool before	Power band	Max torque	Min speed on hot days*	Robust set-point
L	202 min	4.75–7.75 kW	54.0 Nm	1,390 rpm	1,488 rpm, 43.5 Nm
M	202 min	4.75–8.00 kW	58.5 Nm	1,390 rpm	1,500 rpm, 42.5 Nm
H	202 min	4.75–7.75 kW	54.0 Nm	1,390 rpm	1,500 rpm, 42.5 Nm

* when the process/air temperature difference drops to about 8 K. The **robust set-point** is the safe setting farthest from any risky or unseen setting.

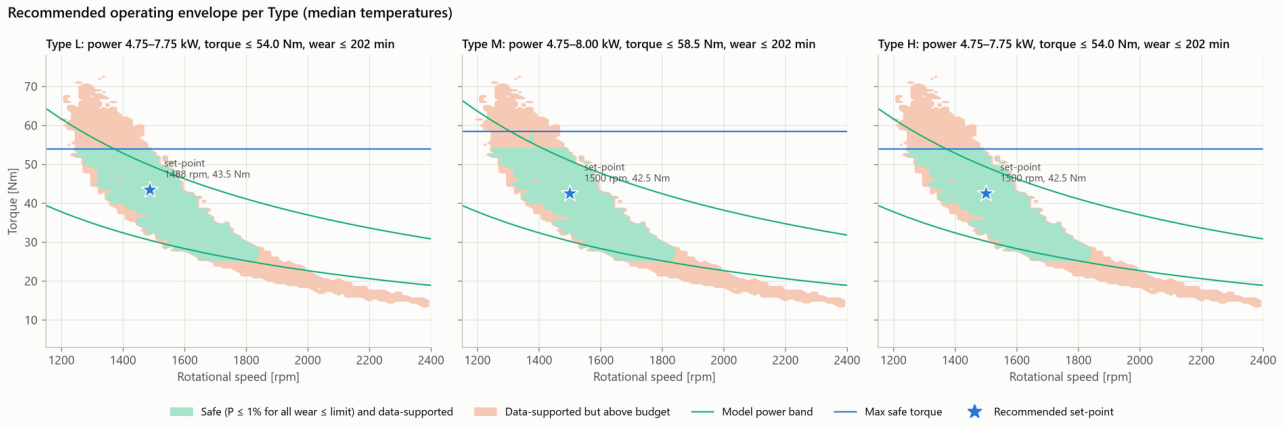


Figure 15. The recommended envelope per Type: green = safe and supported, star = set-point.

The strongest evidence that the model is right: the dataset's failures follow documented rules. We first confirmed those rules in the data (100% agreement), then compared them with what the model learned *without ever seeing the failure flags*:

Mechanism	Model learned	True rule	Verdict
Power failure	safe band 4.75–8.0 kW	fails outside 3.5–9 kW	Inside the band (safely conservative)
Heat dissipation	needs ≥ 1,390 rpm at ΔT ≈ 8 K	fails if ΔT < 8.6 K and speed < 1,380 rpm	Matches
Tool wear	change before 202 min	random failures at 198–253 min	Matches
Overstrain, Type L / M	wear × torque ≤ 10.9k / 11.8k	fails above 11k / 12k	Matches
Overstrain, Type H	≤ 10.9k	fails above 13k	Conservative: only 12 H runs had high wear and torque

8. The code, file by file (in review order)

How the pieces connect: **data.py** loads the table and removes leakage → **features.py** adds physics features → **pipeline.py** chains preprocessing, optional SMOTE and a model → **cv.py** tests every pipeline on the same 5 folds → the notebooks run experiments and save results → **explain.py** / **optimize.py** explain the model and find safe ranges → **app.py** shows everything in the dashboard.

#	File	What it does
1	**README.md**	The whole project on a few pages: problem, the three graded decisions, results, recommendations, how to run and deploy. Read it first.
2	**reports/metrics.md**	Every results table, auto-generated by notebook 02 so it always matches the code: model × strategy, ablation, tuning, leaderboard, selected model, threshold, with the written justification.
3	**notebooks/01_eda.ipynb**	Data checks (shape, types, missing values, column names), class balance, the leakage proof, failure-type counts, distributions, correlations. Ends with a summary table comparing the brief to the real data.
4	**src/data.py**	`load_raw()` loads the CSV (or downloads it from UCI) and renames the columns. `get_X_y()` builds inputs and target and **removes the leakage columns** [GRADED #1].
5	**src/features.py**	`add_physics_features()` computes power, temperature difference and overstrain. Wrapped as a pipeline step. It learns nothing, so it can't leak.
6	**src/pipeline.py**	`build_pipeline(model, strategy, engineered)` returns a ready-to-train imblearn pipeline: [physics features] → encoder/scaler → [SMOTENC] → model. Contains the **SMOTE-in-fold** comment [GRADED #2] and the default settings of the three models.
7	**src/cv.py**	The test harness. `run_cv()` runs the 5 folds and returns per-fold scores plus out-of-fold predictions. Also holds the metric definitions and the **PR-AUC reasoning** [GRADED #3], the nested threshold tuning and the nested hyper-parameter tuning.
8	**notebooks/ 02_modeling.ipynb**	The experiments: SMOTE-in-fold trace, the 18-configuration grid, ablation, PR/ROC curves, the deliberate SMOTE-before-split counter-example, nested tuning, model selection, threshold tuning, saving the final model, writing metrics.md. Runs in ~30 min, mostly tuning.
9	**src/explain.py**	SHAP helpers: runs TreeSHAP on the forest and shows values in real units (K, rpm, Nm) instead of rescaled ones.
10	**src/optimize.py**	The recommendation engine: risk on a speed × torque grid, the data-support mask, tool-wear limit, safe region, power band, robust set-point, and `operating_ranges()`, which produces the final table.
11	**notebooks/ 03_interpret_optimize.ipynb**	Importance, SHAP (global and three individual runs), partial dependence, verification of the physics rules, risk surfaces, tool-wear analysis, budget sensitivity, recommended ranges, comparison with physics, and the five failure-type models used by the dashboard.
12	**app.py**	The Streamlit dashboard (Section 9). It reads saved results and reuses the same src code, so its numbers match the notebooks.
	Skip: src/config.py, src/plots.py	Settings (file paths, column names, the random seed 42) and chart styling only.
	Saved outputs	`models/`: final model, failure-type models, model_meta.json (threshold and settings). `reports/`: all figures (300 dpi), metrics.md, CSVs of fold scores, OOF predictions and operating ranges.

The one code snippet worth memorising

This is how a pipeline is assembled. The SMOTE step only exists for the SMOTE strategy, and imblearn only applies it during training:

```
steps = [("physics", PhysicsFeatures()),            # power, temp diff, overstrain
         ("prep", make_preprocessor(engineered))]  # Type -> 0/1/2, scale numbers
if strategy == "smote":
    steps.append(("smote", SMOTENC(categorical_features=[0])))  # training folds only
steps.append(("clf", make_model(model, strategy)))
return Pipeline(steps)                               # imblearn Pipeline
```

And the test loop. Every fold trains a fresh copy on its training rows and predicts the untouched test rows:

```
for train, test in StratifiedKFold(5, shuffle=True, random_state=42).split(X, y):
    model = clone(pipeline).fit(X.iloc[train], y.iloc[train])   # SMOTE happens here
    p = model.predict_proba(X.iloc[test])[:, 1]                 # never resampled
    scores.append(average_precision_score(y.iloc[test], p))     # PR-AUC
```

9. How to read the dashboard

Start it with `streamlit run app.py`. The top row repeats the four headline numbers: PR-AUC, recall and precision at the alarm threshold, and the threshold itself. Below are four tabs.

Tab	What you see	How to use it
Model performance	The cross-validation table (all models × strategies); a threshold slider; precision-recall and ROC curves; a confusion matrix.	Toggle physics features on/off to see the ablation. Drag the threshold: the black diamond moves along the PR curve and the counts update. At 0.5 you'll see 62 missed and 5 false alarms; at 0.056, 34 missed and ~300 false alarms.
Explainability	SHAP summary, feature importance, partial dependence.	Read with Section 6 of this guide.
What-if simulator	Sliders for Type and the 5 parameters; live failure probability with a status (within budget / elevated / ALARM); the probability of each failure type; power, temperature difference and wear × torque next to their limits; a SHAP chart for this exact setting.	Try: torque 62, wear 215, speed 1,300 gives an overstrain alarm. Air 302.4, process 310.2, speed 1,330 gives heat dissipation. Speed 2,500 with torque 70 shows an "outside the training data" warning.
Operating ranges	The recommendation table, plus an interactive risk map (blue = within the 1% budget, grey = near it, red = above).	Change Type, tool wear and temperatures. Raise tool wear past ~200 min and watch the band turn red. Hover any cell for its exact numbers; the diamond is the recommended set-point.

> **Two things to remember when presenting the dashboard**
>
> (1) Curves and the threshold slider use **out-of-fold** predictions: each run is scored by a model that never saw it. (2) The headline numbers at the top come from fully **nested** CV, the most honest estimate in the project.

10. Honest limitations

- The dataset is **synthetic**: failures follow simple rules. Real machines are noisier, so these exact numbers shouldn't be transferred to real equipment.
- Random failures, the 9 unlabelled failures and the exact timing of tool-wear failures **can't be predicted** from these inputs. This caps recall below 100%.
- At 90% recall, about half the alarms are false. That's the price of catching more failures, and it is shown openly.
- Recommendations hold the other parameters at typical values; the dashboard map explores other conditions.
- SHAP explains the final model, which saw all rows. For individual training rows, the out-of-fold probability is the honest number.
- Model selection compared ~20 candidates on the same folds. The winner's lead (5 of 5 folds) is larger than that effect, but a fresh hold-out dataset would be the next step.

11. Defence cheat-sheet: likely questions

Why not use accuracy?
Always predicting "no failure" scores 96.6% accuracy and catches nothing. With 3.4% failures, accuracy rewards ignoring the problem.

Why PR-AUC instead of ROC-AUC?
PR-AUC focuses on the failure class, and its chance level is 0.034, not 0.5. ROC-AUC divides false alarms by 9,661 healthy runs, so it hides poor precision: logistic regression gets 0.92 ROC-AUC but 0.50 PR-AUC.

Why drop TWF/HDF/PWF/OSF/RNF?
They record which failure happened, so they're only known after it happens. Machine failure is essentially their OR: they alone give 97% recall with no model. Keeping them is leakage.

Where exactly does SMOTE run?
Inside an imblearn Pipeline, so only during fit() and only on the training folds. Test folds are never resampled. Doing it before the split gave a fake 0.999 PR-AUC versus 0.872 done correctly.

Why SMOTENC rather than SMOTE?
Type is categorical. Plain SMOTE would interpolate it into meaningless values like 0.37; SMOTENC assigns a real category.

SMOTE didn't help. Isn't that a problem?
No. It's a finding. Resampling shifts probabilities rather than improving how well failures are ranked. Tuning the threshold achieves the same recall/precision trade-off more transparently.

Why Random Forest over XGBoost?
Higher PR-AUC (0.915 vs 0.875), winning on all 5 paired folds (+0.040 ± 0.014). The lead is larger than the fold-to-fold noise, and the tuned settings were identical in every fold.

What is nested CV and why use it?
A second, inner cross-validation picks hyper-parameters (and the threshold) using training data only, so the outer test folds still give an unbiased score. Without it, tuning makes results look better than they are.

Why threshold 0.056?
We decided in advance to catch at least 90% of failures, because a missed failure costs more than an inspection. 0.056 is the highest threshold that achieves it. It costs precision (52%), which we report openly.

What do the physics features add?
+0.08 to +0.14 PR-AUC for tree models. Failures depend on products like torque × speed; given those directly, trees need one split instead of many.

How do you know the model learned something real?
Without seeing the failure flags, it recovered the dataset's actual failure rules: the power band, the heat-dissipation speed limit, the tool-wear window and the overstrain limits, all within a few percent.

Why a 1% risk budget and not the alarm threshold for ranges?
They answer different questions. The threshold flags individual runs and sits on the ~5% random tool-wear plateau, so it would approve 230 min of wear. The 1% budget asks what keeps risk near normal. Answers are identical for budgets from 0.5% to 2%.

Why only recommend data-supported settings?
Speed and torque are linked, so about 90% of the grid is never observed. There, tree models extrapolate blindly, so recommendations there would be guesses.

Is it reproducible?
Yes. Fixed seed (42), pinned library versions, identical folds for all models. Re-running notebook 02 reproduced every number exactly.

12. Glossary

Term	Meaning
Class imbalance	One outcome (no failure) is far more common than the other (failure).
Leakage	Information in the inputs that wouldn't be available at prediction time, making scores look unrealistically good.
Precision / Recall / F1	Share of alarms that are real / share of failures caught / a balance of the two.
PR-AUC (average precision)	Area under the precision-recall curve. 1 is perfect; chance equals the failure rate (0.034).
ROC-AUC	Area under the true-positive vs false-positive rate curve. Chance is 0.5; flattering under imbalance.
Threshold	Probability above which the model raises an alarm.
Cross-validation (CV), fold	Splitting data into k parts and testing on each in turn. A fold is one part.
Stratified	Each fold keeps the same failure rate as the full data.
Out-of-fold (OOF)	A prediction made by a model that did not train on that row.
Nested CV	CV inside CV: the inner loop tunes, the outer loop scores, so tuning doesn't inflate results.
SMOTE / SMOTENC	Creates synthetic minority examples between real neighbours / version that handles categorical columns.
Class weighting	Makes mistakes on the rare class cost more during training.
Hyper-parameter	A model setting chosen before training (e.g. tree depth).
Pipeline	A chain of steps (features → preprocessing → resampling → model) treated as one model.
SHAP	Splits a single prediction into contributions from each input; they add up to the prediction.
Partial dependence	Average prediction as one input is varied for every row.
Ablation	Removing one component (here, the physics features) to measure its contribution.
Risk budget	The maximum predicted failure risk we accept for a recommended setting (1%).